

# Entrenamiento a Gran Escala

## Módulo 4 · Deep Learning

---

Data/model parallelism, ZeRO, DeepSpeed, mixed precision, gradient checkpointing y estimacion de costes para fine-tuning distribuido.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales  
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

## Entrenamiento a Gran Escala

El preentrenamiento de GPT-3 (175B parámetros) usó unos 10.000 GPUs durante varias semanas y costó aproximadamente 12 millones de dólares. El preentrenamiento de GPT-4 se estima en cientos de millones de dólares. Llama 3 70B se entrenó sobre 15.6 billones de tokens en más de 16.000 GPUs H100 durante meses. Estos son los proyectos de entrenamiento a gran escala que producen los modelos base de los que toda la industria parte.

Pero el entrenamiento a gran escala no es solo prerrogativa de OpenAI y Google. Empresas medianas hacen fine-tuning a escala: entrenar un modelo de 7-70B parámetros sobre decenas de millones de tokens de datos de dominio requiere múltiples GPUs y días de entrenamiento. Entender las técnicas de paralelismo distribuido, los frameworks de entrenamiento a escala, y cómo estimar los costes de cómputo es conocimiento práctico relevante para cualquier organización que quiera construir capacidades de IA propias.

## Los tipos de paralelismo en entrenamiento distribuido

### Data parallelism: escalar el batch size

El paralelismo de datos (Data Parallel, DP) replica el modelo completo en cada GPU y divide el batch de entrenamiento entre ellas. Cada GPU procesa su porción del batch (forward + backward), y los gradientes se sincronizan entre todas las GPUs (allreduce) antes de actualizar los pesos. PyTorch DistributedDataParallel (DDP) es la implementación estándar. Es el tipo de paralelismo más sencillo de implementar y efectivo cuando el modelo cabe en la VRAM de una sola GPU.

### Model parallelism: cuando el modelo no cabe en una GPU

Cuando el modelo es demasiado grande para caber en una sola GPU, el paralelismo de modelo divide el modelo entre GPUs. Pipeline parallelism divide el modelo en etapas secuenciales, cada una en una GPU diferente, con microbatches fluyendo a través del pipeline. Tensor parallelism divide las matrices individuales entre GPUs (cada GPU calcula una parte de la multiplicación matricial y los resultados se combinan). Los modelos muy grandes como GPT-4 usan una combinación de todos estos tipos de paralelismo (3D parallelism).

### ZeRO: reducir la memoria redundante

ZeRO (Zero Redundancy Optimizer, Microsoft DeepSpeed) reduce la memoria requerida por GPU en entrenamiento distribuido eliminando la redundancia en el almacenamiento de parámetros, gradientes y estados del optimizador. ZeRO Stage 1 distribuye los estados del optimizador entre GPUs. ZeRO Stage 2 también distribuye los gradientes. ZeRO Stage 3 también distribuye los parámetros del modelo. Con ZeRO-3 + DeepSpeed, el entrenamiento de modelos de cientos de miles de millones de parámetros se hace factible con clusters de GPUs de tamaño moderado.

---

## Frameworks de entrenamiento a gran escala

### DeepSpeed: el estándar para entrenamiento masivo

DeepSpeed (Microsoft) es el framework de entrenamiento distribuido más adoptado en la industria para modelos grandes. Implementa ZeRO optimization, mixed precision training, gradient checkpointing (re-computar activaciones en backward en lugar de guardarlas, reduciendo memoria a cambio de más cómputo), y pipeline parallelism. La integración con Hugging Face Trainer es simple: solo hay que añadir un fichero de configuración DeepSpeed al Trainer. Meta usó DeepSpeed para entrenar Llama 3.

### Mixed precision training: FP16/BF16 para reducir memoria

El entrenamiento en precisión mixta usa FP16 o BF16 para los forward y backward passes (reduciendo memoria y aumentando velocidad en Tensor Cores) pero mantiene los pesos maestros en FP32 para la actualización de parámetros (evitando problemas de precisión numérica). BF16 es preferido sobre FP16 en hardware moderno (A100, H100) porque tiene mayor rango dinámico y menor riesgo de overflow. La implementación en PyTorch es automática con torch.autocast.

### Gradient checkpointing: memoria vs cómputo

Durante el forward pass, todas las activaciones de cada capa se guardan para el backward pass (se necesitan para calcular los gradientes). Con modelos grandes y secuencias largas, el almacenamiento de todas las activaciones puede consumir más VRAM que los propios pesos del modelo. Gradient checkpointing guarda solo algunas activaciones intermedias y re-computa el resto durante el backward pass, reduciendo el uso de memoria 4-8x a cambio de ~33% más tiempo de entrenamiento.

---

## SECCIÓN 4 · EJEMPLOS REALES

- Preentrenamiento de Llama 3 70B (Meta): entrenado sobre 15.6 billones de tokens usando más de 16.000 GPUs H100 con FSDP (Fully Sharded Data Parallel, la implementación de ZeRO en PyTorch nativo). La infraestructura de entrenamiento usó NVLink para comunicación intra-nodo e InfiniBand NDR para comunicación inter-nodo, maximizando el throughput de comunicación de gradientes.
- Fine-tuning distribuido de Llama 3 8B en 8x H100 (empresa de software): fine-tuning con QLoRA + DeepSpeed ZeRO-2 sobre 500M tokens de código y documentación interna. El entrenamiento completo dura ~18 horas en 8 GPUs H100, produciendo un modelo especializado en el dominio técnico de la empresa con rendimiento superior al modelo genérico en benchmarks internos.
- Databricks AI Runtime: plataforma serverless para entrenamiento distribuido que abstrae la complejidad del paralelismo. Clientes como YipitData y Rivian usan H100s en Databricks para fine-tuning de LLMs y modelos de computer vision sin gestionar la infraestructura de entrenamiento distribuido.

---

## SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Escalar a múltiples GPUs sin optimizar primero en una sola. El entrenamiento distribuido añade complejidad de comunicación (allreduce de gradientes) que puede dominar el tiempo total si el modelo es pequeño. Para modelos que caben en una GPU, el entrenamiento en single-GPU puede ser más rápido que en multi-GPU por la ausencia de overhead de comunicación.

Error 2: No usar gradient accumulation para simular batches grandes. Si el batch size deseado no cabe en VRAM, gradient accumulation acumula los gradientes de N pasos antes de actualizar los pesos, simulando un batch size N veces mayor. Es gratis en términos de tiempo de cómputo y permite trabajar con batch sizes grandes en hardware limitado.

Error 3: Ignorar la eficiencia de comunicación en clusters multi-nodo. En entrenamiento distribuido multi-nodo, el bottleneck suele ser la comunicación de gradientes entre nodos. La elección de la red de interconexión (InfiniBand vs Ethernet) y del algoritmo de allreduce (NCCL, Gloo) tiene un impacto enorme en el throughput real. H100 con NVLink intra-nodo e InfiniBand NDR inter-nodo es la configuración estándar para entrenamientos grandes.

## SECCIÓN 6 · APLICACIÓN PRÁCTICA

---

Para la mayoría de las empresas, el escenario de entrenamiento a gran escala relevante es el fine-tuning multi-GPU de LLMs con QLoRA + DeepSpeed. El proceso: instalar deepspeed y las dependencias, configurar el fichero ds\_config.json (ZeRO Stage 2 o 3 según la memoria disponible, BF16 activado, gradient checkpointing activado), y lanzar con torchrun o accelerate launch. Hugging Face Accelerate es el nivel de abstracción correcto: gestiona la distribución entre GPUs con una API simple que también soporta TPUs y Apple Silicon.

Para estimar el tiempo y coste de un entrenamiento: el número de FLOPs para entrenar un LLM se estima como  $\sim 6 \times N \times D$ , donde N es el número de parámetros y D es el número de tokens. Con H100s a  $\sim 2 \times 10^{15}$  FLOPs/s (considerando eficiencia real  $\sim 30\text{-}50\%$  del peak teórico), puedes estimar el tiempo de entrenamiento y multiplicar por el coste horario de las GPUs.

## SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

---

- M4-T6 (Fine-tuning): el entrenamiento a gran escala es el contexto técnico del fine-tuning distribuido de LLMs.
- M9 (Infraestructura y despliegue): las decisiones de hardware (H100 vs A100, cloud vs on-premise) son directamente relevantes para el coste y la viabilidad del entrenamiento a gran escala.
- M4-T8 (GPUs): las características técnicas de los GPUs (VRAM, bandwidth, NVLink) determinan qué tipo de paralelismo es más eficiente.

## SECCIÓN 8 · RESUMEN EJECUTIVO

---

El entrenamiento a gran escala usa paralelismo distribuido para superar las limitaciones de VRAM de una única GPU: data parallelism (batch dividido entre GPUs), model parallelism (modelo dividido entre GPUs), y ZeRO/FSDP (eliminar redundancia de pesos/gradientes). Mixed precision (BF16) reduce la memoria 2x. Gradient checkpointing

reduce memoria 4-8x a cambio de 33% más tiempo. DeepSpeed y Hugging Face Accelerate son los frameworks estándar. Para fine-tuning empresarial: QLoRA + DeepSpeed ZeRO-2 en 2-8 GPUs H100 es el escenario más común y accesible. Coste orientativo: 8x H100 en AWS ~\$98/hora.